Introduction to c

1)C is a high-level programming language developed in the early 1970s by Dennis Ritchie at Bell Labs. It was designed for system programming and has influenced many other programming languages, including C++, Java, and Python.This language is used for develop system software and operating systems.

PROGRAMMING:- it is an medium which creat communication between use and system.
Based on technology program has been classified into 3 types.
1)Low level
2)Medium level
3)high level

History of c:-
In 1988 c program language is standardized by ANSI.
IN 2000 C program language is standardized by ISO that version is called c-99.

LOW-LEVEL LANGUAGES

1. Definition: Low-level languages are closely related to machine code and provide little abstraction from the hardware.

They allow direct manipulation of memory and hardware resources.

Characteristics:

• Hardware Specific: Low-level languages are often specific to a particular type of hardware architecture.

• Efficiency: They offer high performance and efficiency, as they can directly control hardware resources.

• Complexity: Programming in low-level languages can be complex and error-prone due to the need for detailed management of memory and resources.

Examples:• Assembly Language: A symbolic representation of machine code that is specific to a computer architecture.

Use Cases:

• Operating systems• Embedded systems• Device drivers

2. MEDIUM-LEVEL LANGUAGES

Definition: Medium-level languages provide a balance between low-level and high-level languages.
They offer some abstraction from hardware while still allowing for low-level operations.

Characteristics:

• Portability: Medium-level languages can be more portable than low-level languages, as they are not tied to a specific hardware architecture.

• Control: They provide a good level of control over system resources while still being easier to use than low-level languages.

• Efficiency: They maintain a good balance between performance and ease of use.

Examples:

• C Language: Often considered a medium-level language because it allows for both high-level programming constructs and low-level memory manipulation.

Use Cases:

• System programming• Application development• Game development

USES

C is mainly used for

- 1)Operating systems
- 2)Language compiler
- 3)Database
- 4)Language interpretes
- 5)Utilites
- 6)Network drivers
- 7)assemblers.

FEATURES

- 1)simple
- 2)portability
- 3)powerful
- 4)platform dependent
- 5)structure orenited
- 6)case sensitive
- 7)compiler
- 8)medium level user
- 9)syntax based
- 10)use of pointer.

Source code

Definition: Source code is the human-readable set of instructions written in a programming language.
It is the original code that a programmer writes and can be understood and modified by humans.

Features:•
Human-Readable: Source code is written in a high-level programming language (like C, Python, Java, etc.) that is understandable by programmers.

• Editable: Programmers can easily modify, debug, and enhance the source code.

• Structured: Source code is organized into functions, classes, and modules, making it easier to manage and understand.

• Comments: Programmers can include comments in the source code to explain the logic, making it easier for others (or themselves) to understand later.

Use:• Source code is used during the development phase of software.
It is written, tested, and debugged by programmers

Object Code

Definition: Object code is the machine-readable output generated from the source code after it has been compiled.
It is in binary format and can be executed by a computer's CPU.

Features:

• Machine-Readable: Object code is in a format that the computer can understand and execute directly.
• Not Human-Readable: Unlike source code, object code is not meant to be read or modified by humans.
• Intermediate Representation: Object code may not be a complete executable program; it can be an intermediate step that requires linking with other object files or libraries to create a final executable.
• Platform-Specific: Object code is often specific to the architecture of the machine it was compiled for.

Use:• Object code is used during the execution phase of software. It is what the computer runs to perform the tasks defined in the source code.
• It can be linked with other object files to create an executable program.

Example: The object code generated from the above C source code would be in binary format and not human-readable

KEYWORDS

- It is a pre defined (or) reserved word in c library with a fixed meaning.
- Keywords are used to perform an internal operation. support 32 keywords.
- Every keyword exist in lower case letter like auto,break,case,const,continue ect...
- They are:
- Char, default, do, double, else, enum, extern, float, for, goto, if, int, long, register, return, short, signed, size of, static, struct, switch, union, unsigned, void, volatile, while.

COMMENTS

- Comments are used to provide the description about the logic written in program.
- Comments do not display on output screen
- When we used the comments,then that specific part will be ignorned by compiler
- In “c” language there are 2 types of comments are possible

1)Single line comment
eg: //.....

2)Double line comment
eg: /*.....*/

STRUCTURE OF C PROGRAM

DOCUMENTATION (OPTIONAL)

HEADER FILE

GLOBAL DECLARATION

VOID MAIN ( )

{

LOCAL DECLARATION; (OPTIONAL)

STATEMENTS;

}

DOCUMENTATION

- Like a comment and represent our program
- Eg: /*printing my name*/

HEADER FILES

- Consists of some functions which are already define to compiler.
- Stdio.h, conio.h,string.h,math.h,graphics.h
- Here we can write header file #include.
- #include is a pre-processer directives
- Eg:#include<stdio.h>

GLOBAL DECLARATION

- These are the variables which define outside the main function

MAIN FUNCTION

- Every “c ” program must have atleast one function that is main.

LOCAL DECLARATION

- These variables are which define in inside the main function

STATEMENTS

- 1)INPUT
- 2)OUTPUT

variable

- Variable is an identifier which holds data as another one variable.
- It is an identifier whose value can be changed at the execution time of the program also used to identify input data program .

Variable declaration

- Before you can use a variable in C, you need to declare it. This involves specifying the type of the variable and its name.
- If no input value is given by the user than system will gives a default value called garbage value.
- Syntax:

Variable_name;

inta

Variable initialization

Initialization is the process of assigning a value to a variable at the time of declaration. This can be done in the following ways

:1. Direct Initialization: You can initialize a variable at the time of declaration.

'2. Initialization with Expressions: You can also initialize a variable using an expression.

Eg:int b=40

Variable assignment

1.Assignment refers to giving a variable a value after it has been declared

2.It is a process a value to variable

OPERATORS

1. Arthimetic operators:- These operators are used to perform basic mathematical operations.

- + : Addition
- - : Subtraction
- * : Multiplication
- / : Division
- % : Modulus (remainder of division)

2.RELATIONAL OPERATORS:- RELATIONAL OPERATORSTHESE OPERATORS ARE USED TO COMPARE TWO VALUES.

- == : EQUAL TO
- != : NOT EQUAL TO
- > : GREATER THAN
- < : LESS THAN
- >= : GREATER THAN OR EQUAL TO
- <= : LESS THAN OR EQUAL TO

3. **LOGICAL OPERATORS**:-THESE OPERATORS ARE USED TO COMBINE MULTIPLE CONDITIONS.

- && : LOGICAL AND
- || : LOGICAL OR
- ! : LOGICAL NOT

4. ASSIGNMENT OPERATORS :- ARE USED TO ASSIGN VALUES TO VARIABLES.

- = : SIMPLE ASSIGNMENT
- += : ADD AND ASSIGN
- -= : SUBTRACT AND ASSIGN
- *= : MULTIPLY AND ASSIGN
- /= : DIVIDE AND ASSIGN
- %= : MODULUS AND ASSIGN
- &= : BITWISE AND AND ASSIGN
- |= : BITWISE OR AND ASSIGN
- ^= : BITWISE XOR AND ASSIGN
- <<=: LEFT SHIFT AND ASSIGN
- >>=: RIGHT SHIFT AND ASSIGN

5. BITWISE OPERATORS :-THESE OPERATORS PERFORM OPERATIONS ON BITS AND ARE USED FOR LOW-LEVEL PROGRAMMING.

- & : BITWISE AND
- | : BITWISE OR
- ^ : BITWISE XOR (EXCLUSIVE OR)
- ~ : BITWISE NOT (COMPLEMENT)
- << : LEFT SHIFT• >> : RIGHT SHIFT

6. CONDITIONAL (TERNARY) OPERATOR :-THIS OPERATOR IS A SHORTHAND FOR THE IF-ELSE STATEMENT.

- ? : : SYNTAX: CONDITION ? EXPRESSION1 : EXPRESSION2;
- IF CONDITION IS TRUE, EXPRESSION1 IS EVALUATED; OTHERWISE, EXPRESSION2 IS EVALUATED.

7. INCREMENT AND DECREMENT OPERATORS:-

THE INCREMENT OPERATOR INCREASES THE VALUE OF A VARIABLE BY 1. IT CAN BE USED IN TWO FORMS:

1. PREFIX INCREMENT (++X): THE VARIABLE IS INCREMENTED FIRST, AND THEN ITS VALUE IS USED IN THE EXPRESSION.

2. POSTFIX INCREMENT (X++): THE CURRENT VALUE OF THE VARIABLE IS USED IN THE EXPRESSION FIRST, AND THEN THE VARIABLE IS INCREMENTED

THE DECREMENT OPERATOR DECREASES THE VALUE OF A VARIABLE BY

1. LIKE THE INCREMENT OPERATOR, IT CAN ALSO BE USED IN TWO FORMS:

1. PREFIX DECREMENT (--X): THE VARIABLE IS DECREMENTED FIRST, AND THEN ITS VALUE IS USED IN THE EXPRESSION.

2. POSTFIX DECREMENT (X--): THE CURRENT VALUE OF THE VARIABLE IS USED IN THE EXPRESSION FIRST, AND THEN THE VARIABLE IS DECREMENTED.

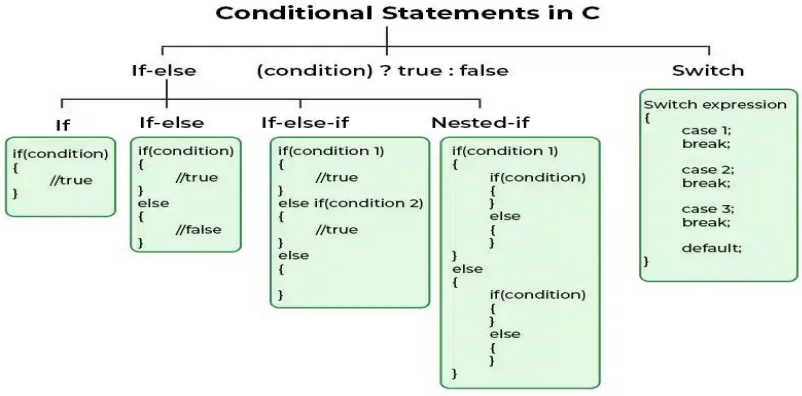
Conditional Statement

- The **conditional statements** (also known as decision control structures) such as if, if else, switch, etc. are used for decision-making purposes in C/C++ programs.
- They are also known as Decision-Making Statements and are used to evaluate one or more conditions and make the decision whether to execute a set of statements or not.
- These decision-making statements in programming languages decide the direction of the flow of program execution.

Need of Conditional Statements

- There come situations in real life when we need to make some decisions and based on these decisions, we decide what should we do next.
- Similar situations arise in programming also where we need to make some decisions and based on these decisions we will execute the next block of code.
- For example, in C if x occurs then execute y else execute z. There can also be multiple conditions like in C if x occurs then execute p, else if condition y occurs execute q, else execute r. This condition of C else-if is one of the many ways of importing multiple conditions.

➔ Types of Conditional Statements in C



1. if

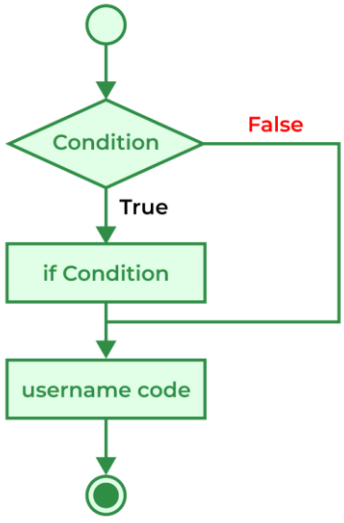
- The if statement is the most simple decision-making statement. It is used to decide whether a certain statement or block of statements will be executed or not i.e if a certain condition is true then a block of statements is executed otherwise not.

Syntax:

```
if
(condition)
{
  // Statements to execute if
  // condition is true
}
```

Here,

- The **condition** after evaluation will be either true or false. C if statement accepts boolean values – if the value is true then it will execute the block of statements below it otherwise not.
- If we do not provide the curly braces '{' and '}' after if(condition) then by default if statement will consider the first immediately below statement to be inside its block.



Example:

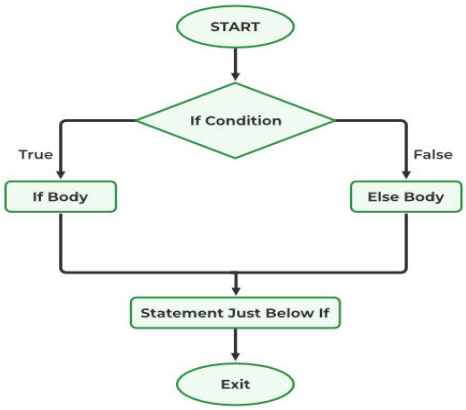
```
#include <stdio.h>

int main()
{

   int gfg = 9;
   // if statement with true condition
   if (gfg < 10) {
      printf("%d is less than 10", gfg);
   }

   // if statement with false condition
   if (gfg > 20) {
      printf("%d is greater than 20", gfg);
   }
return 0;
}
```

Output

```
9 is less than 10
```

2. if-else

- The *if* statement alone tells us that if a condition is true it will execute a block of statements and if the condition is false it won't. But what if we want to do something else when the condition is false?
- Here comes the C *else* statement. We can use the *else* statement with the *if* statement to execute a block of code when the condition is false.



➔Syntax

```
if (condition)
{
  // Executes this block if
   // condition is true
}
else
{
   // Executes this block if
   // condition is false
}
```

Example:

```
#include <stdio.h>
int main()
{
   int i = 20;

   if (i < 15) {
      printf("i is smaller than 15");
   }
   else {
      printf("i is greater than 15");
   }
   return 0;
}
```

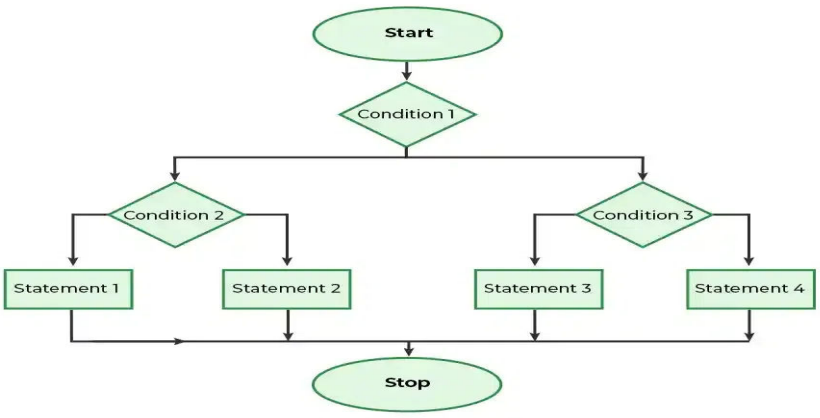
Output: i is greater than 15

3. Nested if-else

- A nested if in C is an if statement that is the target of another if statement.
- Nested if statements mean an if statement inside another if statement. Yes, both C and C++ allow us to nested if statements within if statements, i.e, we can place an if statement inside another if statement.

Syntax

```
if (condition1)
{
  // Executes when condition1 is true
  if (condition2)
  {
    // Executes when condition2 is true
  }
  else
  {
  // Executes when condition2 is false
}
```



Example:

```
// C program to illustrate nested-if statement
#include <stdio.h>
```

```
int main()
{
   int i = 10;

   if (i == 10) {
      // First if statement
      if (i < 15)
         printf("i is smaller than 15\n");

      // Nested - if statement
      // Will only be executed if statement above
      // is true
      if (i < 12)
         printf("i is smaller than 12 too\n");
      else
         printf("i is greater than 15");
   }

   return 0;
}
```

Output:
i is smaller than 15
i is smaller than 12 too

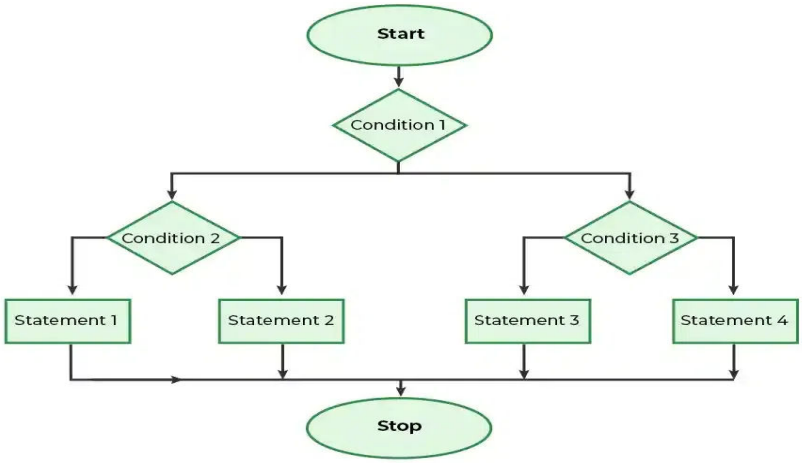
4. if-else-if Ladder

- The if else if statements are used when the user has to decide among multiple options.
- The C if statements are executed from the top down. As soon as one of the conditions controlling the if is true, the statement associated with that if is executed, and the rest of the C else-if ladder is bypassed.

- If none of the conditions is true, then the final else statement will be executed. if-else-if ladder is similar to the switch statement.

Syntax

```
if (condition)
   statement;
else if (condition)
   statement;
.
.

else
   statement;
```



Example

```
// C program to illustrate nested-if statement
#include <stdio.h>
int main()
{
   int i = 20;
```

```
    if (i == 10)
        printf("i is 10");
    else if (i == 15)
        printf("i is 15");
    else if (i == 20)
        printf("i is 20");
    else
        printf("i is not present");
}
```

Output: i is 20

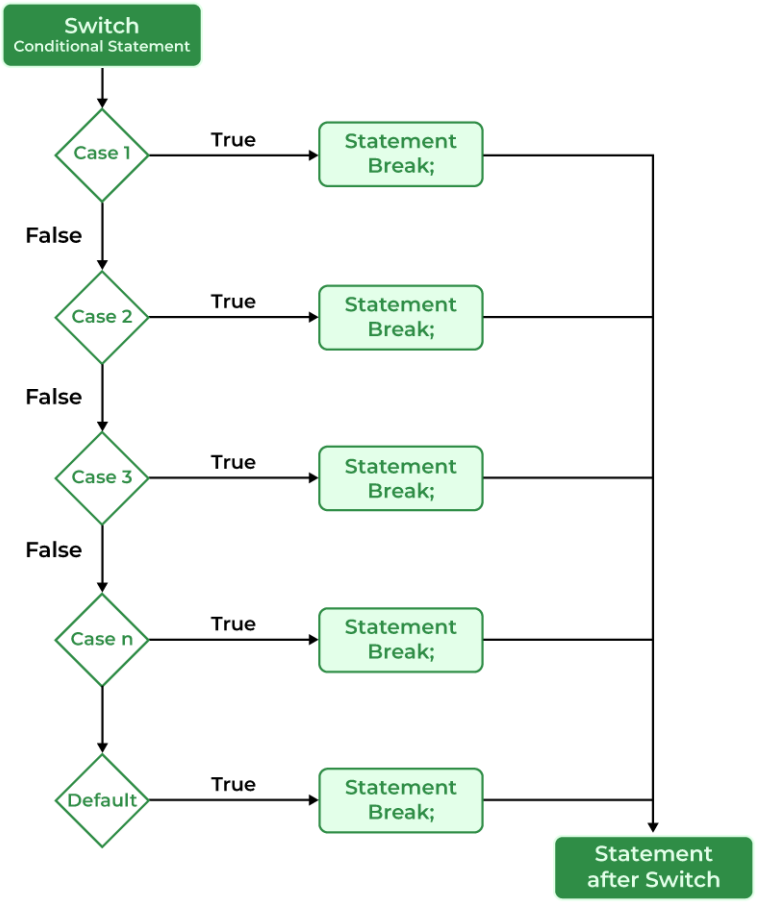
5. Switch Statement

- The switch case statement is an alternative to the if else if ladder that can be used to execute the conditional code based on the value of the variable specified in the switch statement.
- The switch block consists of cases to be executed based on the value of the switch variable.

Syntax

```
switch (expression) {
    case value1:
        statements;
    case value2:
        statements;
    ....
    ....
    ....
    default:
```

Note: *The switch expression should evaluate to either integer or character. It cannot evaluate any other data type.*



Example

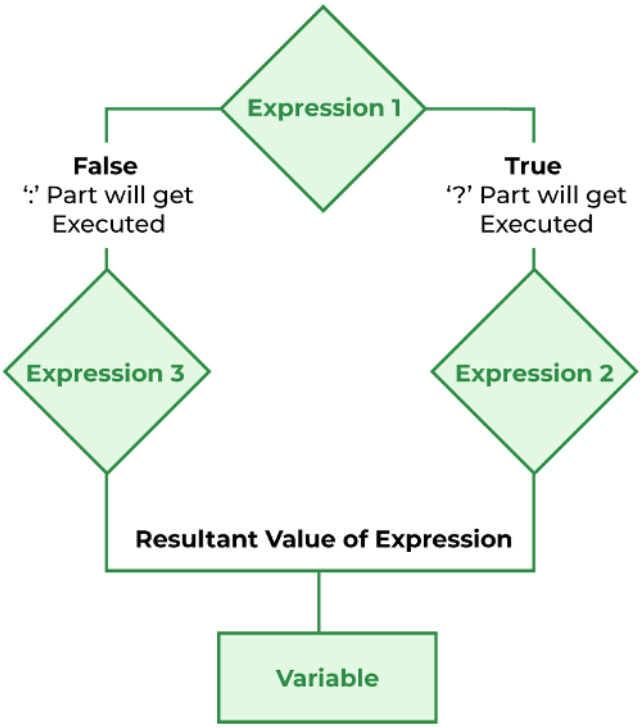
```
// C Program to illustrate the use of switch statement
#include <stdio.h>
int main()
{
   // variable to be used in switch statement
   int var = 2;
   // declaring switch cases
   switch (var) {
   case 1:
      printf("Case 1 is executed");
      break;
   case 2:
      printf("Case 2 is executed");
      break;
```

```
    default:
        printf("Default Case is executed");
        break;
    }
    return 0;
}
```

Output: Case 2 is executed

6. Conditional Operator

- The conditional operator is used to add conditional code in our program. It is like the if-else statement.
- It is also known as the ternary operator as it works on three operands.



Example

```
#include <stdio.h>
// driver code
int main()
{
    int var;
    int flag = 0;
    // using conditional operator to assign the value to var
    // according to the value of flag
    var = flag == 0 ? 25 : -25;
    printf("Value of var when flag is 0: %d\n", var);
    // changing the value of flag
    flag = 1;
   // again assigning the value to var using same statement
    var = flag == 0 ? 25 : -25;
    printf("Value of var when flag is NOT 0: %d", var);

    return 0;
}
```

Output:

```
Value of var when flag is 0: 25
Value of var when flag is NOT 0: -25
```

Loop Satatement in C

- **Loops** in programming are used to repeat a block of code until the specified condition is met.
- A **loop statement** allows programmers to execute a statement or group of statements multiple times without repetition of code.

Example to illustrate need of loop

```
// C program to illustrate need of loops
#include <stdio.h>

int main()
{
   printf  "Hello World\n"
   printf  "Hello World\n"
   printf  "Hello World\n"
   printf  "Hello World\n"
   printf  "Hello World\n"
   printf  "Hello World\n"
   return 0;
}
```

Output:

```
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
```

There are mainly two types of loops in C Programming:

➔ **Entry Controlled loops:** In Entry controlled loops the test condition is checked before entering the main body of the loop. **For Loop and While Loop** is Entry-controlled loops.

➔ **Exit Controlled loops:** In Exit controlled loops the test condition is evaluated at the end of the loop body. The loop body will execute at least once, irrespective of whether the condition is true or false. **do-while Loop** is Exit Controlled loop.

Loop Type	Description
for loop	first Initializes, then condition check, then executes the body and at last, the update is done.
while loop	first Initializes, then condition checks, and then executes the body, and updating can be inside the body.
do-while loop	do-while first executes the body and then the condition check is done.

➢ for Loop

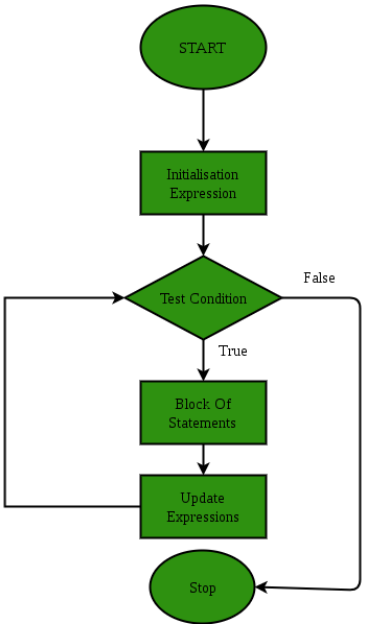
- The **for loop** in C Language provides a functionality/feature to repeat a set of statements a defined number of times.
- The for loop is in itself a form of an **entry-controlled loop**.
- Unlike the while loop and do…while loop, the for loop contains the initialization, condition, and updating statements as part of its syntax. It is mainly used to traverse arrays, vectors, and other data structures.
- “for loop in C programming is a repetition control structure that allows programmers to write a loop that will be executed a specific number of

times. for loop enables programmers to perform n number of steps together in a single line."

Syntax:

```
for (initialize expression; test expression; update expression)
{
   //
   // body of for loop
   //
}
```

- In for loop, a loop variable is used to control the loop. Firstly we initialize the loop variable with some value, then check its test condition.
- If the statement is true then control will move to the body and the body of for loop will be executed. Steps will be repeated till the exit condition becomes true. If the test condition will be false then it will stop.



Example

```
// C program to illustrate for loop
#include <stdio.h>

// Driver code
```

```
int main()
{
  int i = 0;

  for (i = 1; i <= 10; i++)
{
    printf  "Hello World\n"
  }
  return 0;
}
```

Output:

```
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
```

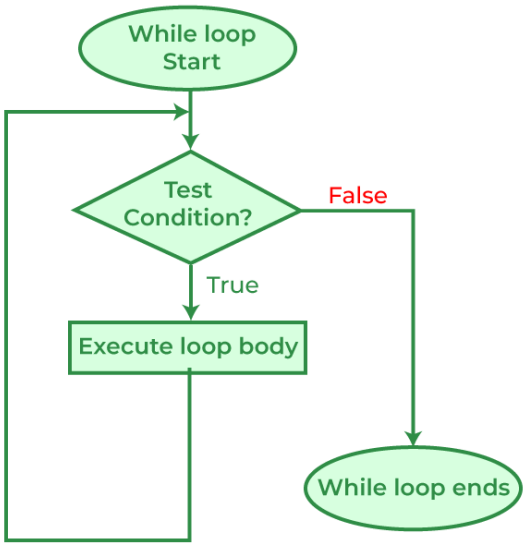
➢ While Loop

- While loop does not depend upon the number of iterations. In for loop the number of iterations was previously known to us but in the While loop, the execution is terminated on the basis of the test condition.
- If the test condition will become false then it will break from the while loop else body will be executed.

Syntax:

```
initialization_expression;

while (test_expression)
```

```
{
 // body of the while loop

   update_expression;
}
```



Example

```
#include <stdio.h>
// Driver code
int main()
{
   int i = 2;
   while(i < 10)
  {
       printf( "Hello World\n");
      i++;
  }
  return 0;
}
```

Output:

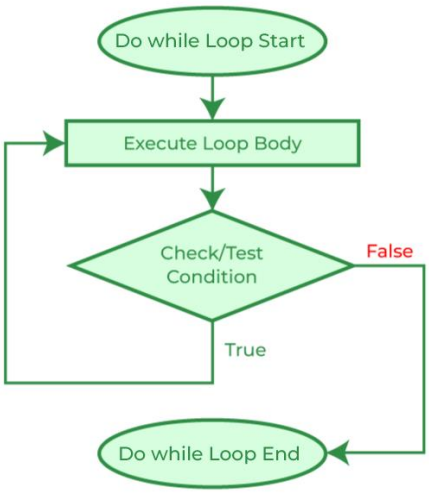
```
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
```

➢ do-while Loop

- The do-while loop is similar to a while loop but the only difference lies in the do-while loop test condition which is tested at the end of the body.
- In the do-while loop, the loop body will **execute at least once** irrespective of the test condition.

Syntax:

```
initialization_expression;
do
{
   // body of do-while loop
   update_expression;
} while (test_expression);
```



Example:

```
#include <stdio.h>

int main()
{
   int i = 2;

  do
  {
    printf( "Hello World\n");

    i++;

  }  while (i < 1);

  return 0;
}
```

Output: Hello World

Above program will evaluate (i<1) as false since i = 2. But still, as it is a do-while loop the body will be executed once.

➢ Infinite Loop

- An infinite loop is executed when the test expression never becomes false and the body of the loop is executed repeatedly.
- A program is stuck in an Infinite loop when the condition is always true.
- Mostly this is an error that can be resolved by using Loop Control statements.

Jump Statement in C

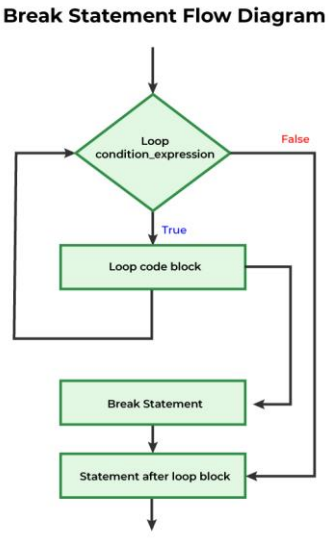
Jump statements are used in C or C++ for the unconditional flow of control throughout the functions in a program.

They support four types of jump statements:

- **Break**
- **Continue**
- **Goto**
- **Return**

➢ Break

- This loop control statement is used to terminate the loop. As soon as the break statement is encountered from within a loop, the loop iterations stop there, and control returns from the loop immediately to the first statement after the loop.
- Basically, break statements are used in situations when we are not sure about the actual number of iterations for the loop or we want to terminate the loop based on some condition.

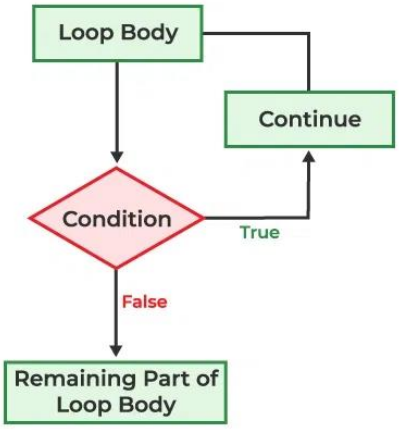


```
#include <stdio.h>
void findElement(int arr[], int size, int key)
{
   // loop to traverse array and search for key
   for (int i = 0; i < size; i++) {
      if (arr[i] == key) {
         printf("Element found at position: %d",
               (i + 1));
         break;
      }
   }
}

int main()
{
   int arr[] = { 1, 2, 3, 4, 5, 6 };

   // no of elements
   int n = 6;

   // key to be searched
   int key = 3;

   // Calling function to find the key
   findElement(arr, n, key);

   return 0;
}
```

Output: Element found at position: 3

➢ Continue

- This loop control statement is just like the break statement.
- The continue statement is opposite to that of the break *statement*, instead of terminating the loop, it forces to execute the next iteration of the loop.
- As the name suggests the continue statement forces the loop to continue or execute the next iteration.
- When the continue statement is executed in the loop, the code inside the loop following the continue statement will be skipped and the next iteration of the loop will begin.




```
#include <stdio.h>

int main()
{
   // loop from 1 to 10
   for (int i = 1; i <= 10; i++) {

            if (i == 6)
         continue;

      else
         // otherwise print the value of i
         printf("%d ", i);
   }
```

```
    return 0;
}
```

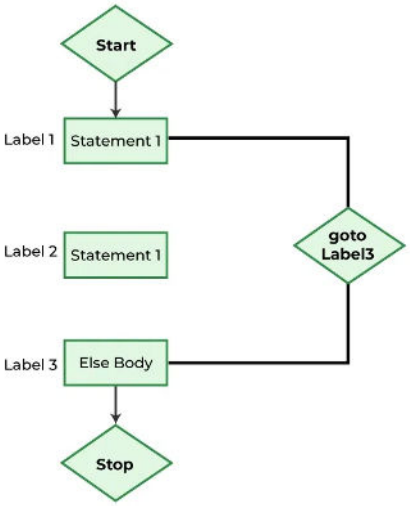
Output: 1 2 3 4 5 7 8 9 10

➢ Goto

- The goto statement in C/C++ also referred to as the unconditional jump statement can be used to jump from one point to another within a function.

Syntax1	**Syntax2**
goto *label;*	*label:*
.	.
.	.
.	.
label:	**goto** *label;*

- In the above syntax, the first line tells the compiler to go to or jump to the statement marked as a label.
- Here, a label is a user-defined identifier that indicates the target statement. The statement immediately followed after 'label:' is the destination statement.
- The 'label:' can also appear before the 'goto label;' statement in the above syntax.

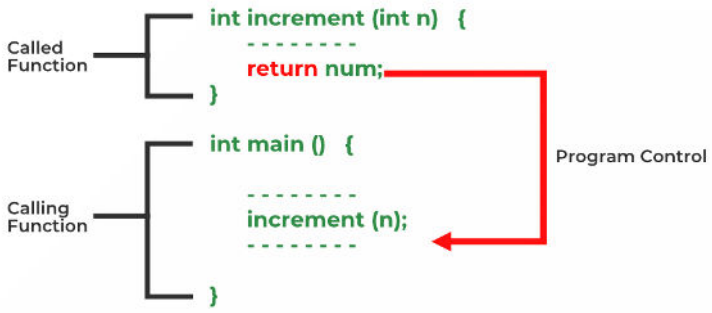


```
#include <stdio.h>
// function to print numbers from 1 to 10
void printNumbers()
{
   int n = 1;
label:
   printf("%d ", n);
   n++;
   if (n <= 10)
      goto label;
}
// Driver program to test above function
int main()
{
   printNumbers();
   return 0;
}
```

Output1 2 3 4 5 6 7 8 9 10

➢ Return

- The return in C or C++ returns the flow of the execution to the function from where it is called. This statement does not mandatorily need any conditional statements.
- As soon as the statement is executed, the flow of the program stops immediately and returns the control from where it was called.
- The return statement may or may not return anything for a void function, but for a non-void function, a return value must be returned.




```
#include <stdio.h>

int SUM(int a, int b)
{
   int s1 = a + b;
   return s1;
}

void Print(int s2)
{
   printf("The sum is %d", s2);
   return;
}

int main()
{
   int num1 = 10;
   int num2 = 10;
   int sum_of = SUM(num1, num2);
   Print(sum_of);
   return 0;
}
```

Output: The sum is 20

Structures

- The structure in C is a user-defined data type that can be used to group items of possibly different types into a single type.
- The **struct keyword** is used to define the structure in the C programming language.
- The items in the structure are called its **member** and they can be of any valid data type.

C Structure Declaration

- We have to declare structure in C before using it in our program. In structure declaration, we specify its member variables along with their datatype.
- We can use the struct keyword to declare the structure in C using the following syntax:

Syntax

```
struct structure_name {
    data_type member_name1;    data_type member_name1;
    ....
    ....
};
```

- The above syntax is also called a structure template or structure prototype and no memory is allocated to the structure in the declaration.

Structure Definition

- To use structure in our program, we have to define its instance. We can do that by creating variables of the structure type.
- We can define structure variables using two methods:

Structure Variable Declaration with Structure Template

```
struct structure_name {
   data_type member_name1;
   data_type member_name1;
   ....
   ....
}variable1, varaible2, ...;
```

- **Structure Variable Declaration after Structure Template**

```
// structure declared beforehandstruct structure_name variable1, variable2, .....;
```

Access Structure Members

- We can access structure members by using the (.) dot operator.

Syntax

```
structure_name.member1;
strcuture_name.member2;
```

In the case where we have a pointer to the structure, we can also use the arrow operator to access the members.

Initialize Structure Members

- Structure members **cannot be** initialized with the declaration. For example, the following C program fails in the compilation.
- We can initialize structure members in 3 ways which are as follows:
  - Using Assignment Operator.
  - Using Initializer List.
  - Using Designated Initializer List.

1. Initialization using Assignment Operator

```
struct structure_name str;
str.member1 = value1;
str.member2 = value2;
str.member3 = value3;
.
.
.
```

2. Initialization using Initializer List

```
struct structure_name str = { value1, value2, value3 };
```

In this type of initialization, the values are assigned in sequential order as they are declared in the structure template.

3. Initialization using Designated Initializer List

```
struct structure_name str = { .member1 = value1, .member2 = value2, .member3 = value3 };
```

Example:

```
#include <stdio.h>

// Define a structure named 'Person'
struct Person {
   char name[50];
   int age;
};
```

```c
int main() {
    // Declare a variable of type 'struct Person'
    struct Person person1;

    // Initialize the values of the structure members
    strcpy(person1.name, "John");
    person1.age = 25;

    // Display the information using the structure
    printf("Name: %s\n", person1.name);
    printf("Age: %d\n", person1.age);

    return 0;
}
```

Output:

```
Name: John
Age: 25
```

typedef for Structures

- The typedef keyword is used to define an alias for the already existing datatype.
- In structures, we have to use the struct keyword along with the structure name to define the variables.
- Sometimes, this increases the length and complexity of the code. We can use the typedef to define some new shorter name for the structure.

Example:

```c
// C Program to illustrate the use of typedef with
// structures
#include <stdio.h>

// defining structure
struct str1 {
        int a;
};

// defining new name for str1
typedef struct str1 str1;

// another way of using typedef with structures
typedef struct str2 {
        int x;
} str2;

int main()
{
        // creating structure variables using new names
        str1 var1 = { 20 };
        str2 var2 = { 314 };

        printf("var1.a = %d\n", var1.a);
        printf("var2.x = %d", var2.x);

        return 0;
}
```

Output:

```
var1.a = 20
var2.x = 314
```

Nested Structures

- C language allows us to insert one structure into another as a member. This process is called nesting and such structures are called nested structures.
- There are two ways in which we can nest one structure into another:

1. Embedded Structure Nesting

In this method, the structure being nested is also declared inside the parent structure.

2. Separate Structure Nesting

In this method, two structures are declared separately and then the member structure is nested inside the parent structure.

Structure Pointer in C

We can define a pointer that points to the structure like any other variable. Such pointers are generally called Structure Pointers. We can access the members of the structure pointed by the structure pointer using the (->) arrow operator.

Example:

```
#include <stdio.h>

struct Student {
    int rollNumber;
    char name[50];
    float marks;
};

int main() {
    // Declare a structure variable
    struct Student student1;

    // Initialize structure fields using dot operator
    student1.rollNumber = 101;
    strcpy(student1.name, "John");
    student1.marks = 85.5;

    // Declare a structure pointer and assign address of student1 to it
    struct Student *ptr = &student1;

    // Access structure fields using the structure pointer and arrow operator
    printf("Roll Number: %d\n", ptr->rollNumber);
    printf("Name: %s\n", ptr->name);
    printf("Marks: %.2f\n", ptr->marks);
    return 0;
}
```

Output:

```
Roll Number: 101
Name: John
Marks: 85.50
```

Uses of Structure in C

C structures are used for the following:

- The structure can be used to define the custom data types that can be used to create some complex data types such as dates, time, complex numbers, etc. which are not present in the language.
- It can also be used in data organization where a large amount of data can be stored in different fields.
- Structures are used to create data structures such as trees, linked lists, etc.
- They can also be used for returning multiple values from a function.

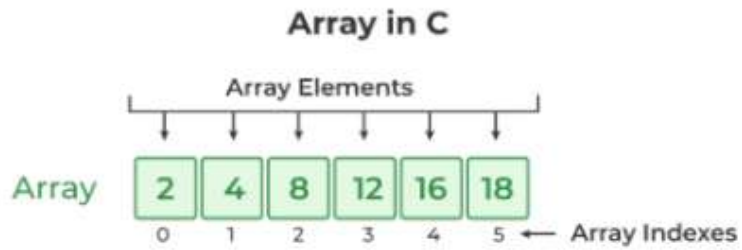
Limitations of C Structures

In C language, structures provide a method for packing together data of different types. A Structure is a helpful tool to handle a group of logically related data items. However, C structures also have some limitations.

- **Higher Memory Consumption:** It is due to structure padding.
- **No Data Hiding:** C Structures do not permit data hiding. Structure members can be accessed by any function, anywhere in the scope of the structure.
- **Functions inside Structure:** C structures do not permit functions inside the structure so we cannot provide the associated functions.
- **Static Members:** C Structure cannot have static members inside its body.
- **Construction creation in Structure:** Structures in C cannot have a constructor inside Structures.

Array in C

- **Array in C** is one of the most used data structures in C programming. It is a simple and fast way of storing multiple values under a single name. In this article, we will study the different aspects of array in C language such as array declaration, definition, initialization, types of arrays, array syntax, advantages and disadvantages, and many more.
- An array in C is a fixed-size collection of similar data items stored in contiguous memory locations.
- It can be used to store the collection of primitive data types such as int, char, float, etc., and derived and user-defined data types such as pointers, structures, etc.

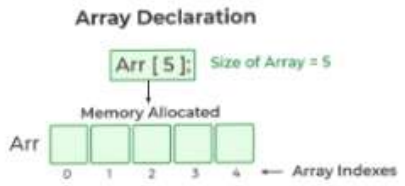


Declaration of Array

- In C, we must declare the array like any other variable before using it. We can declare an array by specifying its name, the type of its elements, and the size of its dimensions.
- When we declare an array in C, the compiler allocates the memory block of the specified size to the array name.

Syntax :

```
data_type array_name [size];
      or
data_type array_name [size1] [size2]...[sizeN];
```



- The C arrays are static in nature, i.e., they are allocated memory at the compile time.

Example

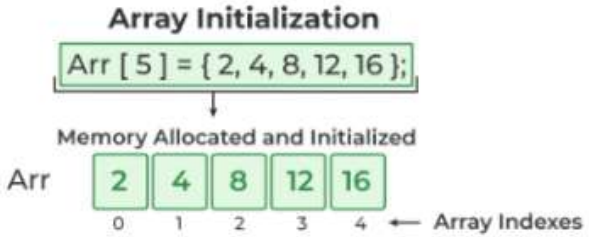
```
// C Program to illustrate the array declaration
#include <stdio.h>

int main()
{

// declaring array of integers
int arr_int[5];
// declaring array of characters
char arr_char[5];

return 0;
}
```

Initialization of Array

- Initialization in C is the process to assign some initial value to the variable.
- When the array is declared or allocated memory, the elements of the array contain some garbage value.

➢ Array Initialization with Declaration

- In this method, we initialize the array along with its declaration. We use an initializer list to initialize multiple elements of the array.
- An initializer list is the list of values enclosed within braces **{ }** separated b a comma.



➢ Array Initialization with Declaration without Size

- If we initialize an array using an initializer list, we can skip declaring the size of the array as the compiler can automatically deduce the size of the array in these cases.
- The size of the array in these cases is equal to the number of elements present in the initializer list as the compiler can automatically deduce the size of the array.

➢ Array Initialization after Declaration (Using Loops)

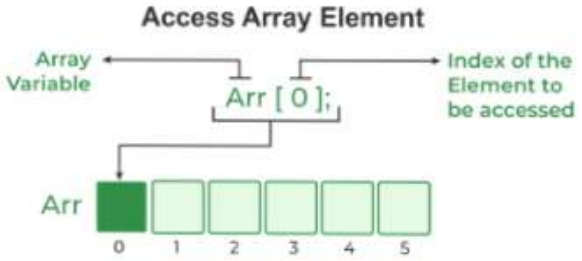
- We initialize the array after the declaration by assigning the initial value to each element individually.
- We can use for loop, while loop, or do-while loop to assign the value to each element of the array.

Example

```
// C Program to demonstrate array initialization
#include <stdio.h>

int main()
{

// array initialization using initialier list
int arr[5] = { 10, 20, 30, 40, 50 };

// array initialization using initializer list without specifying size
int arr1[] = { 1, 2, 3, 4, 5 };

// array initialization using for loop
float arr2[5];
for (int i = 0; i < 5; i++) {
arr2[i] = (float)i * 2.1;
}
return 0;
}
```

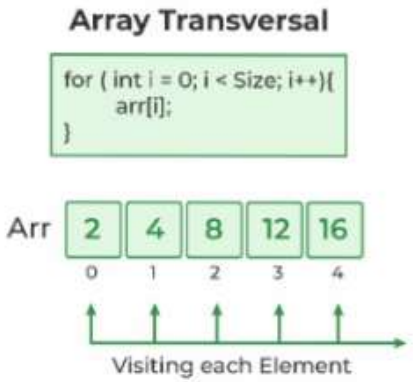
Access Array Elements

- We can access any element of an array in C using the array subscript operator **[ ]** and the index value ***i*** of the element.
- One thing to note is that the indexing in the array always starts with 0, i.e., the **first element** is at index **0** and the **last element** is at **N – 1** where **N** is the number of elements in the array.



Traversal of Array

- Traversal is the process in which we visit every element of the data structure.
- For C array traversal, we use loops to iterate through each element of the array.



How to use Array in C ?

The following program demonstrates how to use an array in the C programming language:

```
// C Program to demonstrate the use of array
#include <stdio.h>
int main()
{
// array declaration and initialization
int arr[5] = { 10, 20, 30, 40, 50 };
// modifying element at index 2
arr[2] = 100;
// traversing array using for loop
printf("Elements in Array: ");
for (int i = 0; i < 5; i++) {
printf("%d ", arr[i]);
}
return 0;
```

```
}
```

Output: Elements in Array: 10 20 100 40 50

Types of Array

There are two types of arrays based on the number of dimensions it has. They are as follows:

1. **One Dimensional Arrays (1D Array)**
2. **Multidimensional Arrays**

1. One Dimensional Array

The One-dimensional arrays, also known as 1-D arrays in C are those arrays that have only one dimension.

→ Syntax

```
array_name [size];
```

Example:

```
#include <stdio.h>
int main()
{
// 1d array declaration
int arr[5];

// 1d array initialization using for loop
for (int i = 0; i < 5; i++) {
arr[i] = i * i - 2 * i + 1;
}
printf("Elements of Array: ");
// printing 1d array by traversing using for loop
for (int i = 0; i < 5; i++) {
printf("%d ", arr[i]);
}
return 0;
}
```

Output: Elements of Array: 1 0 1 4 9

2. Multidimensional Array

Multi-dimensional Arrays in C are those arrays that have more than one dimension. Some of the popular multidimensional arrays are 2D arrays and 3D arrays. We can declare arrays with more dimensions than 3d arrays but they are avoided as they get very complex and occupy a large amount of space.

- **Two-Dimensional Array**

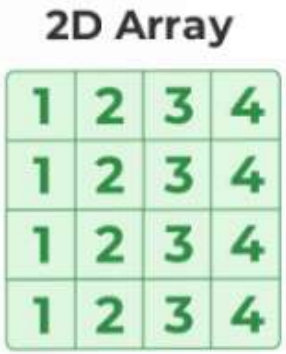
  A Two-Dimensional array or 2D array in C is an array that has exactly two dimensions. They can be visualized in the form of rows and columns organized in a two-dimensional plane.

  → Syntax

  ```
  array_name[size1] [size2];
  ```

  Here,

  - **size1:** Size of the first dimension.
  - **size2:** Size of the second dimension.



Example of 2D Array

```
// C Program to illustrate 2d array
#include <stdio.h>

int main()
{

// declaring and initializing 2d array
int arr[2][3] = { 10, 20, 30, 40, 50, 60 };

printf("2D Array:\n");
// printing 2d array
for (int i = 0; i < 2; i++) {
for (int j = 0; j < 3; j++) {
printf("%d ",arr[i][j]);
}
printf("\n");
}

return 0;
}
```

```
Output:
2D Array:
10 20 30
40 50 60
```

2.Example:

```
// C Program to print the elements of a Two-Dimensional array

#include <stdio.h>

int main(void)
{
// an array with 3 rows and 2 columns.
int x[3][2] = { { 0, 1 }, { 2, 3 }, { 4, 5 } };

// output each array element's value
for (int i = 0; i < 3; i++) {
for (int j = 0; j < 2; j++) {
printf("Element at x[%i][%i]: ", i, j);
printf("%d\n", x[i][j]);
}
}

return (0);
}
```

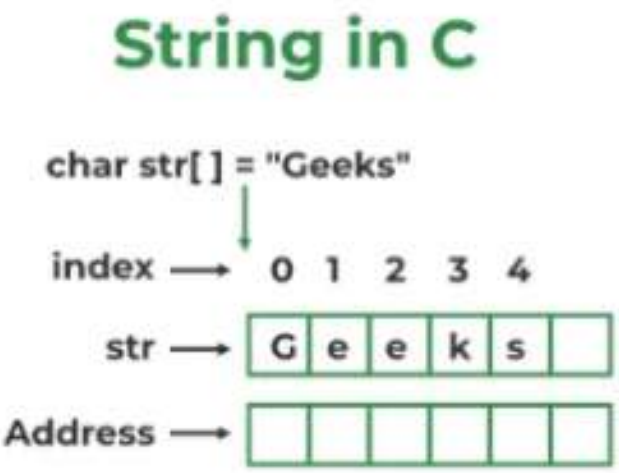
```
Output:
Element at x[0][0]: 0
Element at x[0][1]: 1
Element at x[1][0]: 2
Element at x[1][1]: 3
Element at x[2][0]: 4
Element at x[2][1]: 5
```

➢ Advantage of Array

- In an array, accessing an element is very easy by using the index number.
- The search process can be applied to an array easily.
- 2D Array is used to represent matrices.
- For any reason a user wishes to store multiple values of similar type then the Array can be used and utilized efficiently.
- Arrays have low overhead.
- C provides a set of built-in functions for manipulating arrays, such as sorting and searching.
- C supports arrays of multiple dimensions, which can be useful for representing complex data structures like matrices.
- Arrays can be easily converted to pointers, which allows for passing arrays to functions as arguments or returning arrays from functions.

String in C

- A **String** in C programming is a sequence of characters terminated with a null character '\0'. The C String is stored as an array of characters.
- The difference between a character array and a C string is the string is terminated with a unique character '\0'.



C String Declaration Syntax

Declaring a string in C is as simple as declaring a one-dimensional array.

Syntax

```
char string_name[size];
```

- In the above syntax **str_name** is any name given to the string variable and size is used to define the length of the string, i.e the number of characters strings will store.
- There is an extra terminating character which is the **Null character ('\0') used to indicate the termination of a string that differs strings from normal character arrays.**

Initilization of String

We can initialize a C string in 4 different ways which are as follows:

1. Assigning a string literal without size

String literals can be assigned without size. Here, the name of the string str acts as a pointer because it is an array.

```
char str[] = "Myselfkritika";
```

2. Assigning a string literal with a predefined size

String literals can be assigned with a predefined size. But we should always account for one extra space which will be assigned to the null character. If we want to store a string of size n then we should always declare a string with a size equal to or greater than n+1.

```
char str[50] = "MyselfKritika";
```

3. Assigning character by character with size

We can also assign a string character by character. But we should remember to set the end character as '\0' which is a null character.

```
char str[14] = { 'M','y','s','e','l','f','K','r','i','t','i','k','a','\0'};
```

4. Assigning character by character without size

We can assign character by character without size with the NULL character at the end. The size of the string is determined by the compiler automatically.

```
char str[] = { 'M','y','s','e','l','f','K','r','i','t','i','k','a','\0'};
```

Notes:

- When a Sequence of characters enclosed in the double quotation marks is encountered by the compiler, a null character '\0' is appended at the end of the string by default.
- After declaration, if we want to assign some other text to the string, we have to assign it one by one or use built-in strcpy() function because the direct assignment of string literal to character arrray is only possible in declration.

The C arrays are static in nature, i.e., they are allocated memory at the compile time.

Example:

```c
// C Program to illustrate string
#include <stdio.h>
#include <string.h>

int main()
{
   // declare and initialize string
   char str[] = "Kritika";

   // print string
   printf("%s\n", str);

   int length = 0;
   length = strlen(str);

   // displaying the length of string
   printf("Length of string str is %d", length);

   return 0;
}
```

Output:

```
Kritika
Length of string str is 7
```

- We can see in the above program that strings can be printed using normal printf statements just like we print any other variable. Unlike arrays, we do not need to print a string, character by character.

➢ **Note:** The C language does not provide an inbuilt data type for strings but it has an access specifier "**%s**" which can be used to print and read strings directly.

How to Read a String Separated by Whitespaces in C?

We can use multiple methods to read a string separated by spaces in C. The two of the common ones are:

→ We can use the fgets() function to read a line of string and gets() to read characters from the standard input (stdin) and store them as a C string until a newline character or the End-of-file (EOF) is reached.
→ We can also scanset characters inside the scanf() function

Example:

```
// C program to illustrate fgets()
#include <stdio.h>
#define MAX 50
int main()
{
char str[MAX];

// MAX Size if 50 defined
fgets(str, MAX, stdin);

printf("String is: \n");

// Displaying Strings using Puts
puts(str);

return 0;
}
```

Input:

```
Wearedoingcode
```

Output:

```
String is:
Wearedoingcode
```

C String Length

The length of the string is the number of characters present in the string except for the NULL character. We can easily find the length of the string using the loop to count the characters from the start till the NULL character is found.

Array of Strings in C

In C programming String is a 1-D array of characters and is defined as an array of characters. But an array of strings in C is a two-dimensional array of character types. Each String is terminated with a null character (\0). It is an application of a 2d array.

→ **Syntax:**

```
char variable_name[r] = {list of string};
```

Here,

- **var_name** is the name of the variable in C.
- **r** is the maximum number of string values that can be stored in a string array.
- **c** is the maximum number of character values that can be stored in each string array.
- to avoid high space consumption in our program we can use an Array of Pointers in C.

Pointers

- **Pointers** are one of the core components of the C programming language. A pointer can be used to store the **memory address** of other variables, functions, or even other pointers.
- The use of pointers allows low-level memory access, dynamic memory allocation, and many other functionality in C.
- A pointer is defined as a derived data type that can store the address of other C variables or a memory location. We can access and manipulate the data stored in that memory location using pointers.
- As the pointers store the memory addresses, their size is independent of the type of data they are pointing to.

➔ Syntax

The syntax of pointers is similar to the variable declaration in C, but we use the **(*) dereferencing operator** in the pointer declaration.

```
datatype * ptr;
```

Where,

- **ptr** is the name of the pointer.
- **datatype** is the type of data it is pointing to.

The above syntax is used to define a pointer to a variable. We can also define pointers to functions, structures, etc.

How to Use Pointers?

The use of pointers can be divided into three steps:

- **Pointer Declaration**
- **Pointer Initialization**
- **Dereferencing**

➢ Pointer Declaration

- In pointer declaration, we only declare the pointer but do not initialize it. To declare a pointer, we use the **(*) dereference operator** before its name.
- The pointer declared here will point to some random memory address as it is not initialized. Such pointers are called wild pointers.

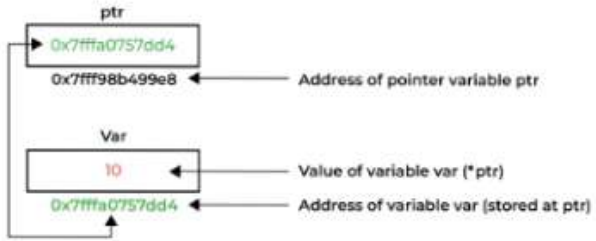
➢ Pointer Initialization

- Pointer initialization is the process where we assign some initial value to the pointer variable.
- We generally use the **(&) addressof operator** to get the memory address of a variable and then store it in the pointer variable.
- We can also declare and initialize the pointer in a single step. This method is called **pointer definition** as the pointer is declared and initialized at the same time.

Note: *It is recommended that the pointers should always be initialized to some value before starting using it. Otherwise, it may lead to number of errors.*

➢ Dereferencing

- Dereferencing a pointer is the process of accessing the value stored in the memory address specified in the pointer.
- We use the same **(*) dereferencing operator** that we used in the pointer declaration.



Example:

```
#include <stdio.h>


int main() {
   int num = 10;  // Declare an integer variable
   int *ptr;     // Declare a pointer to an integer


   ptr = &num;   // Assign the address of 'num' to the pointer
   printf("Value of num: %d\n", num);
   printf("Address of num: %p\n", &num);
   printf("Value of ptr: %p\n", ptr);
   printf("Value pointed by ptr: %d\n", *ptr);
   return 0;
}
```

Output:

```
Value of num: 10
Address of num: 0x7ffd2a8c234c
Value of ptr: 0x7ffd2a8c234c
Value pointed by ptr: 10
```

Types of Pointers

Pointers can be classified into many different types based on the parameter on which we are defining their types.

If we consider the type of variable stored in the memory location pointed by the pointer, then the pointers can be classified into the following types:

1. Integer Pointers

- As the name suggests, these are the pointers that point to the integer values.
- These pointers are pronounced as **Pointer to Integer.**
- Similarly, a pointer can point to any primitive data type. It can point also point to derived data types such as arrays and user-defined data types such as structures.

2. Array Pointer

- Pointers and Array are closely related to each other. Even the array name is the pointer to its first element. They are also known as Pointer to Arrays.

3. Structure Pointer

- The pointer pointing to the structure type is called Structure Pointer or Pointer to Structure.
- It can be declared in the same way as we declare the other primitive data types.
- In C, structure pointers are used in data structures such as linked lists, trees, etc.

4. Function Pointers

- Function pointers point to the functions. They are different from the rest of the pointers in the sense that instead of pointing to the data, they point to the code.
- Let's consider a function prototype – **int func (int, char)**

Note: The syntax of the function pointers changes according to the function prototype.

5. Double Pointers

- In C language, we can define a pointer that stores the memory address of another pointer. Such pointers are called double-pointers or pointers-to-pointer.

6. NULL Pointer

- The Null Pointers are those pointers that do not point to any memory location. They can be created by assigning a NULL value to the pointer.
- A pointer of any type can be assigned the NULL value.
- It is said to be good practice to assign NULL to the pointers currently not in use.

7. Void Pointer

- The Void pointers in C are the pointers of type void. It means that they do not have any associated data type.
- They are also called **generic pointers** as they can point to any type and can be typecasted to any type.

8. Wild Pointers

- The Wild Pointers are pointers that have not been initialized with something yet.
- These types of C-pointers can cause problems in our programs and can eventually cause them to crash.

9. Constant Pointers

- In constant pointers, the memory address stored inside the pointer is constant and cannot be modified once it is defined.
- It will always point to the same memory address.

10. Pointer to Constant

- The pointers pointing to a constant value that cannot be modified are called pointers to a constant.
- Here we can only access the data pointed by the pointer, but cannot modify it. Although, we can change the address stored in the pointer to constant.

Pointer Arithmetic

The Pointer Arithmetic refers to the legal or valid arithmetic operations that can be performed on a pointer.

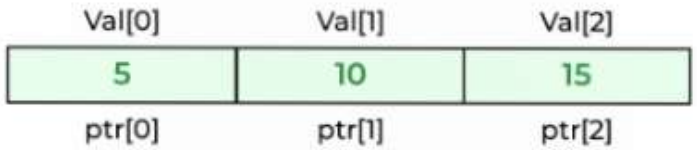
It is slightly different from the ones that we generally use for mathematical calculations as only a limited set of operations can be performed on pointers. These operations include:

- Increment in a Pointer
- Decrement in a Pointer
- Addition of integer to a pointer
- Subtraction of integer to a pointer
- Subtracting two pointers of the same type
- Comparison of pointers of the same type.
- Assignment of pointers of the same type.

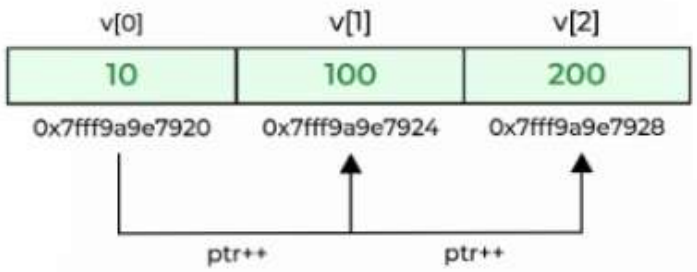
C Pointers and Arrays

- In C programming language, pointers and arrays are closely related. An array name acts like a pointer constant.
- The value of this pointer constant is the address of the first element. For example, if we have an array named val then **val** and **&val[0]** can be used interchangeably.
- If we assign this value to a non-constant pointer of the same type, then we can access the elements of the array using this pointer.

Example 1: Accessing Array Elements using Pointer with Array Subscript



Example 2: Accessing Array Elements using Pointer Arithmetic



➔ Advantages of Pointers

- Pointers are used for dynamic memory allocation and deallocation.
- An Array or a structure can be accessed efficiently with pointers
- Pointers are useful for accessing memory locations.
- Pointers are used to form complex data structures such as linked lists, graphs, trees, etc.
- Pointers reduce the length of the program and its execution time as well.

➔ Disadvantages of Pointers

- Memory corruption can occur if an incorrect value is provided to pointers.
- Pointers are a little bit complex to understand.
- Pointers are majorly responsible for memory leaks in C.
- Pointers are comparatively slower than variables in C.
- Uninitialized pointers might cause a segmentation fault.

Functions

- A **function in C** is a set of statements that when called perform some specific task.
- It is the basic building block of a C program that provides modularity and code reusability.
- The programming statements of a function are enclosed within **{ } braces**, having certain meanings and performing certain operations. They are also called subroutines or procedures in other languages.

Syntax of Functions in C

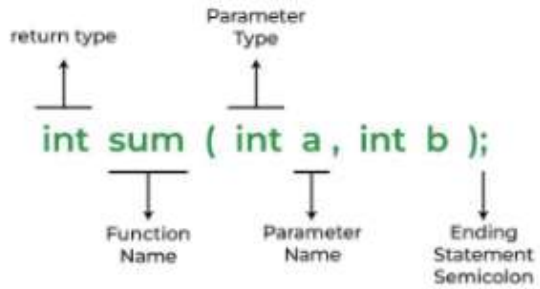
The syntax of function can be divided into 3 aspects:

- ★ **Function Declaration**
- ★ **Function Definition**
- ★ **Function Calls**

1.Function Declarations

- In a function declaration, we must provide the function name, its return type, and the number and type of its parameters.
- A function declaration tells the compiler that there is a function with the given name defined somewhere else in the program.

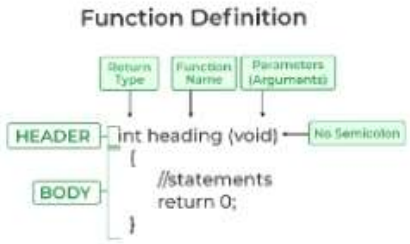
Example:



Note: *A function in C must always be declared globally before calling it.*

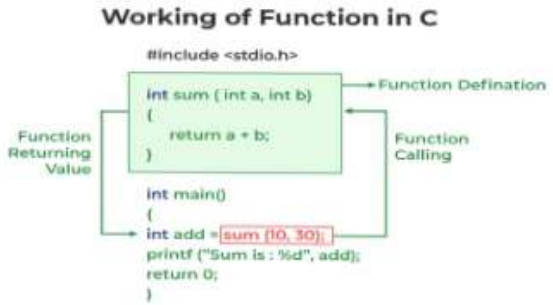
2.Function Definition

- The function definition consists of actual statements which are executed when the function is called (i.e. when the program control comes to the function).
- A C function is generally defined and declared in a single step because the function definition always starts with the function declaration so we do not need to declare it explicitly.



3.Function Call

- A function call is a statement that instructs the compiler to execute the function. We use the function name and parameters in the function call.
- In the below example, the first sum function is called and 10,30 are passed to the sum function. After the function call sum of a and b is returned and control is also returned back to the main function of the program.



Note: *Function call is neccessary to bring the program control to the function definition. If not called, the function statements will not be executed.*

Example:

```
// C program to show function call and definition
#include <stdio.h>

// Function that takes two parameters  a and b as inputs

 int sum(int a, int b)
{
  return a + b;
}

// Driver code
int main()
{
   int add = sum(10, 30);

  printf("Sum is: %d", add);
  return 0;
}
```

Output: Sum is: 40

As we noticed, we have not used explicit function declaration. We simply defined and called the function.

Function Return Type

- Function return type tells what type of value is returned after all function is executed. When we don't want to return a value, we can use the void data type.

Note: *Only one value can be returned from a C function. To return multiple values, we have to use pointers or structures.*

Function Arguments

Function Arguments (also known as Function Parameters) are the data that is passed to a function.

Conditions of Return Types and Arguments

In C programming language, functions can be called either with or without arguments and might return values. They may or might not return values to the calling functions.

1) Function with no arguments and no return value
2) Function with no arguments and with return value
3) Function with argument and with no return value
4) Function with arguments and with return value

How Does C Function Work?

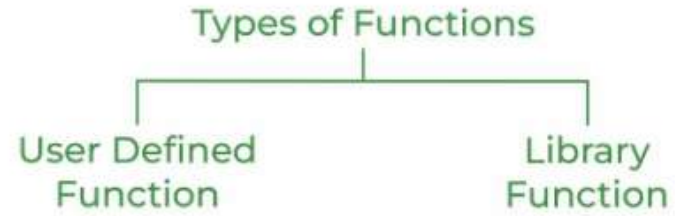
Working of the C function can be broken into the following steps as mentioned below:

1. **Declaring a function:** Declaring a function is a step where we declare a function. Here we define the return types and parameters of the function.
2. **Defining a function**
3. **Calling the function:** Calling the function is a step where we call the function by passing the arguments in the function.
4. **Executing the function:** Executing the function is a step where we can run all the statements inside the function to get the final result.
5. **Returning a value:** Returning a value is the step where the calculated value after the execution of the function is returned. Exiting the function is the final step where all the allocated memory to the variables, functions, etc is destroyed before giving full control to the main function.

Types of Functions

There are two types of functions in C:

1. **Library Functions**
2. **User Defined Functions**



1. Library Function

- A library function is also referred to as a **"built-in function"**. A compiler package already exists that contains these functions, each of which has a specific meaning and is included in the package.
- Built-in functions have the advantage of being directly usable without being defined, whereas user-defined functions must be declared and defined before being used.

Example:

pow(), sqrt(), strcmp(), strcpy() etc.

Advantages of C library functions

- C Library functions are easy to use and optimized for better performance.
- C library functions save a lot of time i.e, function development time.
- C library functions are convenient as they always work.

Example:

```
// C program to implement the above approach
#include <math.h>
#include <stdio.h>
// Driver code
int main()
{
  double Number;
  Number = 49;
  // Computing the square root with the help of predefined C library function

double squareRoot = sqrt(Number);

  printf("The Square root of %.2lf = %.2lf",
        Number, squareRoot);
  return 0;
}
```

Output: The Square root of 49.00 = 7.00

2. User Defined Function

- Functions that the programmer creates are known as User-Defined functions or **"tailor-made functions"**. User-defined functions can be improved and modified according to the need of the programmer.
- Whenever we write a function that is case-specific and is not defined in any header file, we need to declare and define our own functions according to the syntax.
- A **user-defined function** is a type of function in C language that is defined by the user himself to perform some specific task. It provides code reusability and modularity to our program.
- User-defined functions are different from built-in functions as their working is specified by the user and no header file is required for their usage.

Advantages of User-Defined Functions

- Changeable functions can be modified as per need.
- The Code of these functions is reusable in other programs.
- These functions are easy to understand, debug and maintain.

Example:

```
// C program to show user-defined functions
#include <stdio.h>

int sum(int a, int b)
{
  return a + b;
}

// Driver code
int main()
{
  int a = 30, b = 40;

// function call
  int res = sum(a, b);

  printf("Sum is: %d", res);
  return 0;
}
```

Output: Sum is: 70

➔ Passing Parameters to functions

We can pass parameters to a function in C using two methods:

1. **Call by Value**
2. **Call by Reference**

1. Call by value

- In call by value, a copy of the value is passed to the function and changes that are made to the function are not reflected back to the values.
- Actual and formal arguments are created in different memory locations.

Example:

```c
// C program to show use of call by value
#include <stdio.h>

void swap(int a, int b)
{
   int temp = a;
   a = b;
   b = temp;
}

// Driver code
int main()
{
   int x = 10, y = 20;
   printf("Values of x and y before swap are: %d, %d\n", x, y);
   swap(x, y);
   printf("Values of x and y after swap are: %d, %d", x,y);
   return 0;
}
```

Output:
Values of x and y before swap are: 10, 20
Values of x and y after swap are: 10, 20

Note: *Values aren't changed in the call by value since they aren't passed by reference.*

2. Call by Reference

- In a call by Reference, the address of the argument is passed to the function, and changes that are made to the function are reflected back to the values.
- We use the pointers of the required type to receive the address in the function.

```c
// C program to implement Call by Reference
#include <stdio.h>

void swap(int* a, int* b)
{
   int temp = *a;
   *a = *b;
   *b = temp;
}

// Driver code
int main()
{
   int x = 10, y = 20;
   printf("Values of x and y before swap are: %d, %d\n", x,y);
   swap(&x, &y);
   printf("Values of x and y after swap are: %d, %d", x,y);
   return 0;
}
```

Output:
Values of x and y before swap are: 10, 20
Values of x and y after swap are: 20, 10

Main Function in C

- The main function is an integral part of the programming languages such as C, C++, and Java.

- The **main** function in C is the entry point of a program where the execution of a program starts.
- It is a user-defined function that is mandatory for the execution of a program because when a C program is executed, the operating system starts executing the statements in the **main()** function.

Advantages of Functions in C

Functions in C is a highly useful feature of C with many advantages as mentioned below:

- The function can reduce the repetition of the same statements in the program.
- The function makes code readable by providing modularity to our program.
- There is no fixed number of calling functions it can be called as many times as you want.
- The function reduces the size of the program.
- Once the function is declared you can just use it without thinking about the internal working of the function.

Disadvantages of Functions in C

The following are the major disadvantages of functions in C:

- Cannot return multiple values.
- Memory and time overhead due to stack frame allocation and transfer of program control.

Union

- **Union** can be defined as a user-defined data type which is a collection of different variables of different data types in the same memory location.
- The union can also be defined as many members, but only one member can contain a value at a particular point in time.
- **Union** is a user-defined data type, but unlike structures, they share the same memory location.

```
struct abc
{
  int a;
  char b;
}
```

- In the above code is the user-defined structure that consists of two members, i.e., 'a' of type **int** and 'b' of type **character**.
- When we check the addresses of 'a' and 'b', we found that their addresses are different.
- Therefore, we conclude that the members in the structure do not share the same memory location.
- When we define the union, then we found that union is defined in the same way as the structure is defined but the difference is that union keyword is used for defining the union data type, whereas the struct keyword is used for defining the structure.
- The union contains the data members, i.e., 'a' and 'b', when we check the addresses of both the variables then we found that both have the same addresses. It means that the union members share the same memory location.

Why do we need C unions?

Consider one example to understand the need for C unions. Let's consider a store that has two items:

- Books
- Shirts

- Store owners want to store the records of the above-mentioned two items along with the relevant information. For example, Books include Title, Author, no of pages, price, and Shirts include Color, design, size, and price. The 'price' property is common in both items. The Store owner wants to store the properties, then how he/she will store the records.

Difference between C Structure and C Union

The following table lists the key difference between the structure and union in C:

Structure	Union
The size of the structure is equal to or greater than the total size of all of its members.	The size of the union is the size of its largest member.
The structure can contain data in multiple members at the same time.	Only one member can contain data at the same time.
It is declared using the struct keyword.	It is declared using the union keyword.

Let's, see the difference with the help of example:

```
#include <stdio.h>
union unionJob
{
  //defining a union
  char name[32];
  float salary;
  int workerNo;
```

```
} uJob;

struct structJob
{
  char name[32];
  float salary;
  int workerNo;
} sJob;

int main()
{
  printf("size of union = %d bytes", sizeof(uJob));
  printf("\nsize of structure = %d bytes", sizeof(sJob));
  return 0;
}
```

Output:

```
size of union = 32
size of structure = 40
```

File Handling in C

- File handing in C is the process in which we create, open, read, write, and close operations on a file.
- C language provides different functions such as fopen(), fwrite(), fread(), fseek(), fprintf(), etc. to perform input, output, and many different C file operations in our program.

Why do we need File Handling in C?

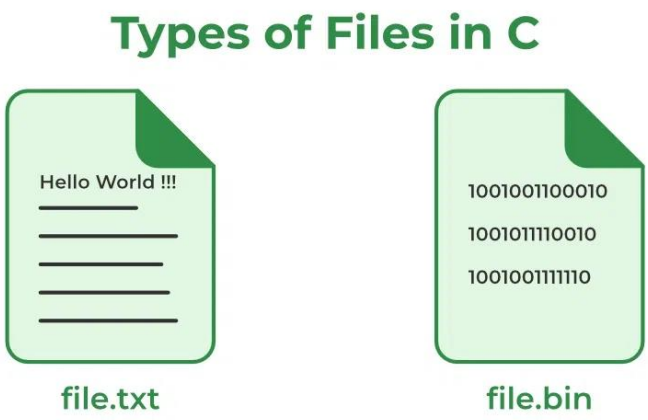
- So far the operations using the C program are done on a prompt/terminal which is not stored anywhere.
- The output is deleted when the program is closed. But in the software industry, most programs are written to store the information fetched from the program.
- The use of file handling is exactly what the situation calls for.

Features of using file

- **Reusability:** The data stored in the file can be accessed, updated, and deleted anywhere and anytime providing high reusability.
- **Portability:** Without losing any data, files can be transferred to another in the computer system. The risk of flawed coding is minimized with this feature.
- **Efficient:** A large amount of input may be required for some programs. File handling allows you to easily access a part of a file using few instructions which saves a lot of time and reduces the chance of errors.
- **Storage Capacity:** Files allow you to store a large amount of data without having to worry about storing everything simultaneously in a program.

Types of Files in C

1. **Text Files**
2. **Binary Files**



1. Text Files

A text file contains data in the **form of ASCII characters** and is generally used to store a stream of characters.

- Each line in a text file ends with a new line character ('\n').
- It can be read or written by any text editor.
- They are generally stored with **.txt** file extension.
- Text files can also be used to store the source code.

2. Binary Files

A binary file contains data in **binary form (i.e. 0's and 1's)** instead of ASCII characters. They contain data that is stored in a similar manner to how it is stored in the main memory.

- The binary files can be created only from within a program and their contents can only be read by a program.
- More secure as they are not easily readable.
- They are generally stored with **.bin** file extension.

Functions for file handling

No.	Function	Description
1	fopen()	opens new or existing file
2	fprintf()	write data into the file
3	fscanf()	reads data from the file
4	fputc()	writes a character into the file
5	fgetc()	reads a character from file
6	fclose()	closes the file
7	fseek()	sets the file pointer to given position
8	fputw()	writes an integer to file
9	fgetw()	reads an integer from file
10	ftell()	returns current position
11	rewind()	sets the file pointer to the beginning of the file

Opening File: fopen()

We must open a file before it can be read, write, or update. The fopen() function is used to open a file.

Syntax:

```
FILE *fopen( const char * filename, const char * mode );
```

The fopen() function accepts two parameters:

- The file name (string). If the file is stored at some specific location, then we must mention the path at which the file is stored. For example, a file name can be like **"c://some_folder/some_file.ext"**.
- The mode in which the file is to be opened. It is a string.

Closing File: fclose()

The fclose() function is used to close a file. The file must be closed after performing all the operations on it.

Syntax:

```
int fclose( FILE *fp );
```

Writing File : fprintf() function

The fprintf() function is used to write set of characters into file. It sends formatted output to a stream.

Syntax:

```
int fprintf(FILE *stream, const char *format [, argument, ...])
```

Reading File : fscanf() function

The fscanf() function is used to read set of characters from file. It reads a word from the file and returns EOF at the end of file.

Syntax:

```
int fscanf(FILE *stream, const char *format [, argument, ...])
```